Lab 3: Architecture Selection & Justification

Topic: Dropshipping Inventory & Order Orchestrator

NAME : ROHITH S

SRN : PES2UG24CS409

SEC : G

Architecture Selection

We chose an **Event-Driven Layered Microservices Architecture** (optimized into a decoupled Event-Driven Orchestration model) for the **Dropshipping Inventory & Order Orchestrator System**.

Reason 1 — Clear separation of concerns for asynchronous multi-vendor integration

Unlike a single-warehouse system, a dropshipping platform coordinates independent entities: E-commerce Storefronts (Shopify, WooCommerce), Third-Party Logistics/Suppliers (3PL webhooks, supplier APIs), and Operational Managers. The requirements naturally decouple into three core operational layers: **Presentation Layer** (Storefront Webhooks, Admin Web Interface, Supplier Dashboards), **Business Layer / Event Orchestrator** (Order Routing Engine, Supplier Inventory Sync, Real-time Stock Allocation), and **Data Layer** (Unified Inventory Ledger, Order Lifecycle DB, Supplier Catalog Mappings). This structure ensures high modularity, allowing independent integration of new supplier APIs or store sales channels without disturbing core business rules or database integrity.

Reason 2 — Adapts to the scale and asynchronous nature of dropshipping operations

In dropshipping, order volumes fluctuate violently (flash sales, marketing surges), and supplier API response times vary widely. A tight client-server or monolithic design would suffer from blocking calls and cascading failures if a single supplier API goes offline. By combining a layered structure with asynchronous event orchestration (Message Queue / Event Bus), incoming sales orders from e-commerce storefronts are immediately ingested, acknowledged, and placed in a processing queue. This guarantees resilient order routing and automated retry mechanisms without blocking user-facing checkout flows.

Security Advantage

The **Presentation Layer (External Storefronts & Supplier Webhooks)** never interacts with the **Data Layer** directly. Every order payload and stock adjustment request must pass through an API Gateway and the Business / Order Orchestration Layer, where OAuth2 authentication, HMAC webhook signature verification, payload validation, and Role-Based Access Control (RBAC) are strictly enforced (e.g., restricting margin configuration and supplier payout logs to Admin roles). Funneling all multi-vendor traffic through strict business-layer gatekeepers minimizes the attack surface, prevents SQL injection/data tampering, and protects sensitive supplier pricing data.

Performance Benefit

Dropshipping platforms must maintain accurate virtual stock levels across 100,000+ SKUs sourced from multiple supplier feeds to prevent overselling. By decoupling inventory ingestion into an asynchronous caching pipeline, the **Inventory Sync Component** maintains an in-memory Redis cache of global stock counts backed by the primary database. Lookups and reservation checks

complete in **under 50 ms**, seamlessly meeting strict latency requirements during peak traffic while ensuring supplier feed sync operations run asynchronously in the background.

Architectural Diagram & Component Layout

Below is the architectural representation adapted for the Dropshipping Inventory & Order Orchestrator System:

